Gebze Technical University

CSE241 OBJECT ORIENTED PROGRAMMING

-

PA6 REPORT

Veysel Cemaloğlu

200104004107

Report on Programming Assignment 6

Introduction

This report outlines the design and implementation of a class hierarchy in Java, as required by the assignment. The task involves categorizing objects into visual and non-visual, as well as playable and non-playable, and implementing an observer pattern to manage updates in a media dataset.

Task Description

The assignment required creating a class hierarchy to represent different types of media objects, such as text, audio, video, and image. These objects needed to be categorized based on their visual and playable characteristics. The goal was to design a system where these objects could be stored in a dataset, and media players and viewers could interact with them using the observer pattern.

Compiling and Running

Compile and run the code:

```
make
```

Solution Approach

1. **Class Hierarchy Design**

   The design begins with a base class Entry, which serves as the superclass for all media types. The following classes extend from Entry:

   - **Text:** Represents text media, implementing ‘Non_visual’ and ‘Non_playable’.
   - **Audio:** Represents audio media, implementing ‘Non_visual’ and ‘Playable’.
   - **Video:** Represents video media, implementing ‘Visual’ and ‘Playable’.
   - **Image:** Represents image media, implementing ‘Visual’ and ‘Non_playable’.

2. **Interface and Abstract Class Implementation**

   Interfaces were created to define the capabilities of different media types:

   - **Visual:** Indicates that a media object has visual properties.
   - **Playable:** Indicates that a media object can be played.
   - **Non_visual:** Indicates that a media object does not have visual properties.
   - **Non_playable:** Indicates that a media object cannot be played.

   The ‘Observer’ is an abstract class that defines methods for handling updates in the dataset.

3. **Observer Pattern**

   The observer pattern was implemented to allow media players (Player class) and viewers (Viewer class) to subscribe to updates in the dataset (Dataset class). The Observer interface was used to define the update mechanism.

4. **Simulation**

   A simulation was set up in the Test class to create a dataset, add various media objects to it, and demonstrate the interaction between media players, viewers, and the dataset.

UML Design

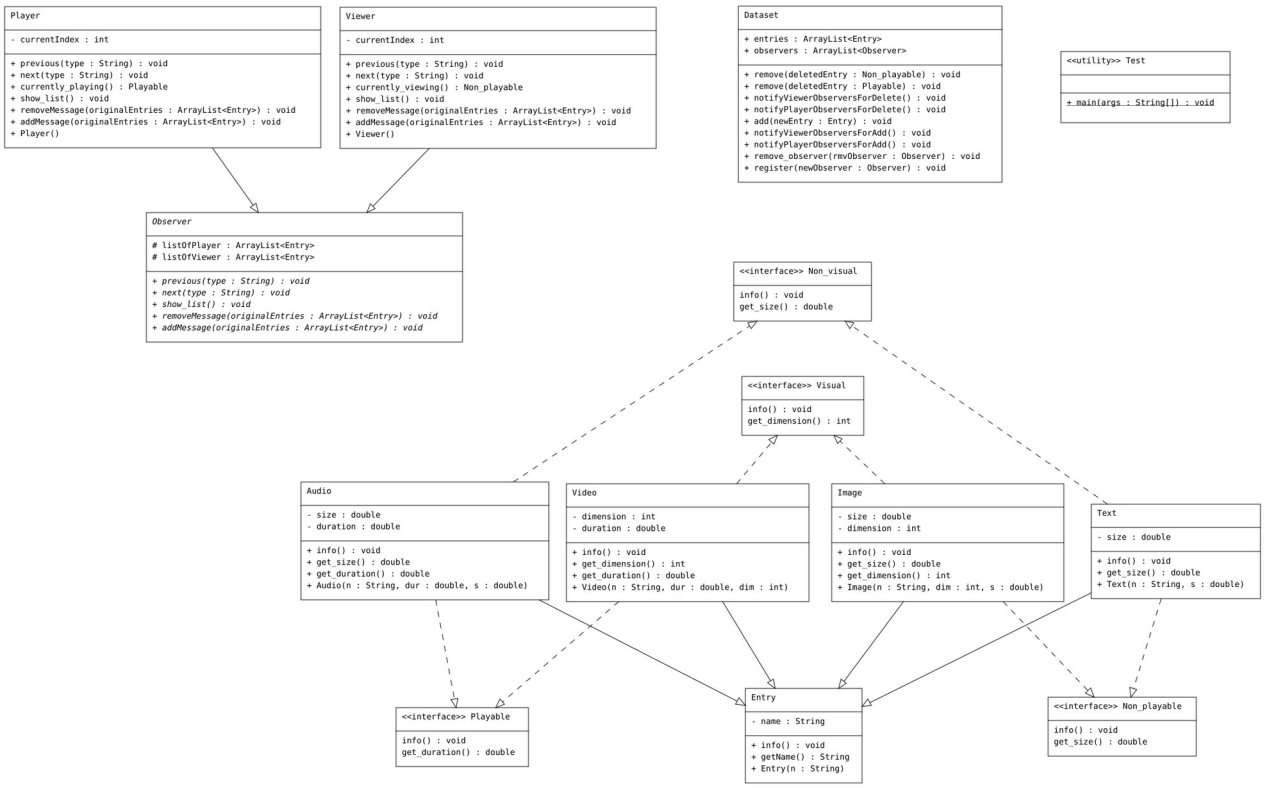
The UML diagram for this solution includes the following elements:

1. **Classes**:
   - Entry: Base class for all media objects.
   - Text: Derived class representing text, implementing Non_visual and Non_playable.
   - Audio: Derived class representing audio, implementing Non_visual and Playable.
   - Video: Derived class representing video, implementing Visual and Playable.
   - Image: Derived class representing image, implementing Visual and Non_playable.
   - Dataset: Class representing a collection of media objects.
   - Player: Class representing a media player that plays playable objects.
   - Viewer: Class representing a viewer that shows non-playable objects.

2. **Interfaces**:
   - Visual
   - Playable
   - Non_visual
   - Non_playable

3. **Abstract Classes:**
   - Observer

The diagram shows the relationships between these classes and interfaces, illustrating how they interact within the system. I added this 'uml.diagram.png' to the zip file in case you want to take a closer look.



Conclusion

The assignment successfully demonstrates the application of object-oriented principles, including inheritance, interfaces, and design patterns, to solve a complex problem. The use of the observer pattern ensures that the system is extendable and maintains a clear separation of concerns.